

Context-Aware Dynamic Token Mapper

An AI-Assisted Adaptive Authorization Extension for Keycloak

Extending Open-Source Identity and Access Management Infrastructure through
Contextual Risk-Based Protocol Mapping

Document Type	Internship Project Concept Note
Target Platform	Keycloak Open-Source IAM Server
Domain	Identity and Access Management (IAM), Adaptive Authorization
Status	Proposed / For Review
By	Bereket Kiros – Mekelle University

1. Project Title

Context-Aware Dynamic Token Mapper: An AI-Assisted Adaptive Authorization Extension for Keycloak

2. Background

Keycloak is a widely adopted open-source Identity and Access Management (IAM) platform that provides authentication, authorization, single sign-on (SSO), and identity federation services based on open standards such as OAuth 2.0, OpenID Connect (OIDC), and SAML 2.0. At the core of its OIDC implementation lies the Protocol Mapper framework, an extensibility mechanism that allows developers to control how user, client, and session attributes are translated into claims within issued tokens (ID Tokens, Access Tokens, and UserInfo responses).

Modern access control is increasingly moving away from static, role-based models toward adaptive and risk-aware authorization, where the privileges granted during a session can be influenced by contextual signals such as network origin, device fingerprint, time of access, and behavioral history. Standards bodies and IAM vendors alike now recognize context-aware access as a core capability of mature Zero Trust architectures. However, Keycloak's default Protocol Mapper implementations are largely static: they map stored attributes to claims without evaluating the real-time context of the authentication event.

This project proposes to extend Keycloak's token generation pipeline through a custom Protocol Mapper that consults an external, independently developed AI Risk Assessment Service at token-issuance time, and uses the returned risk classification to adjust the claims of the issued token, without altering any persisted identity data in Keycloak's database.

3. Problem Statement

Keycloak's authorization model, by default, issues tokens whose claims and embedded privileges are determined almost entirely by the user's stored roles, groups, and client scope configuration at the time of login. This model does not natively account for the risk profile of the specific authentication event. Consequently, a login from an unfamiliar device, an atypical geographic location, or an unusual time of day is granted the same set of privileges as a routine, low-risk login from a trusted device and network.

This creates a gap between Keycloak's static, identity-centric authorization model and the contextual, risk-aware access decisions that modern security postures require. Organizations that wish to implement adaptive authorization currently need to build such logic outside Keycloak, at the application layer, which duplicates access-control logic, is inconsistent across services, and is disconnected from the token issuance process itself, where enforcement is most efficient and auditable.

There is, therefore, a need for an extension mechanism, implemented within Keycloak's own extension points, that can incorporate contextual risk signals into the token issuance decision itself, while preserving the integrity and stability of the user's persisted identity record.

4. Motivation

- **Architectural learning opportunity:** Working within Keycloak's Service Provider Interface (SPI) and Protocol Mapper framework provides deep, practical exposure to a production-grade, standards-compliant IAM codebase, its extension points, and its token generation lifecycle.
- **Relevance to Zero Trust and adaptive security:** Contextual, risk-based access control is a foundational concept in Zero Trust architecture. Implementing it as a Keycloak extension demonstrates how established IAM platforms can be evolved to support modern security requirements without being replaced.
- **Separation of concerns as a design principle:** The project deliberately separates identity data management (Keycloak), risk scoring (an external AI service), and claim transformation (the custom mapper), reflecting sound software architecture practice and avoiding unnecessary coupling between IAM infrastructure and analytical/ML components.
- **Non-destructive authorization:** By modifying only the issued token and never the stored user profile, the design keeps the system auditable, reversible per session, and safe to pilot in a production-like environment without risk of permanent data corruption.
- **Open-source contribution value:** The resulting mapper is designed to be a self-contained, reusable extension that could plausibly be proposed to or adapted for the Keycloak community, reinforcing good open-source engineering discipline (testing, documentation, configuration-driven behavior).

5. Objectives

5.1 General Objective

To design, implement, and evaluate a custom OIDC Protocol Mapper extension for Keycloak that dynamically adjusts issued JWT token claims based on contextual risk information obtained from an external AI Risk Assessment Service, without modifying any persisted user identity data.

5.2 Specific Objectives

1. To study and document Keycloak's internal architecture, authentication pipeline, and Protocol Mapper (SPI) framework relevant to OIDC token generation.
2. To design a custom Java class extending `AbstractOIDCProtocolMapper` capable of intercepting the token-building process for ID Tokens and/or Access Tokens.
3. To define a context-collection mechanism that gathers authentication-time signals (e.g., IP address, login timestamp, device/user-agent characteristics, and available login history) from the Keycloak authentication session.
4. To design and implement a lightweight external AI Risk Assessment Service (Python/FastAPI) that accepts contextual authentication data and returns a risk score or risk classification.
5. To implement a secure, resilient integration (HTTP/REST) between the Keycloak mapper and the external risk service, including timeout and fallback handling.

6. To implement claim-transformation logic that, based on the returned risk classification, conditionally removes privileged scopes/roles from the token, adds a `risk_level` claim, and/or inserts a restricted indicator claim.
7. To verify, through testing, that the user's stored roles and profile in the Keycloak database remain unchanged before and after the process, confirming that only the issued token is affected.
8. To document the extension's configuration, deployment, and operational behavior for maintainability and potential reuse.

6. Proposed Solution

The proposed solution introduces a custom OIDC Protocol Mapper that plugs into Keycloak's existing, standards-based extension mechanism, rather than modifying Keycloak's core authentication or token-signing logic directly. The solution is composed of three cleanly separated responsibilities, described in detail in Section 7.

At a high level, the mapper is invoked during the token-building phase of the authentication flow, immediately before a token is signed. At that point, it gathers available contextual signals from the authentication session, forwards them to the external AI Risk Assessment Service over HTTP, and receives a risk classification in return. The mapper then applies a deterministic, configurable transformation to the token claims being assembled for that session:

- Removal or suppression of specified privileged scopes or roles from the issued token's claim set for high-risk sessions.
- Insertion of a `risk_level` claim (e.g., LOW, MEDIUM, HIGH) reflecting the assessed risk classification.
- Insertion of a restricted boolean or status claim indicating that the session is operating under reduced privileges.

Crucially, none of these transformations touch Keycloak's persistent storage layer. The user's stored roles, groups, and attributes in the Keycloak database remain exactly as they were before authentication; only the in-memory token representation being built for that specific session is altered. This preserves data integrity, keeps the behavior strictly session-scoped, and allows the mechanism to be safely disabled or reconfigured without any data migration or cleanup.

7. High-Level System Overview

The system is organized as three loosely coupled components communicating over well-defined interfaces, which allows each to be developed, tested, and reasoned about independently.

Component	Responsibility	Explicitly Does NOT
Keycloak Core	Handles authentication, session management, identity storage, and token signing; exposes the SPI/Protocol Mapper extension point and invokes the custom mapper during token building.	Does not perform risk scoring; does not know about the AI service's internal logic.
Custom Protocol Mapper	Collects contextual session data, calls the AI Risk Assessment Service, interprets the response, and applies claim-level transformations to the token being built for that session.	Does not compute risk itself; does not write to the Keycloak database; does not alter stored user roles or profile.
AI Risk Assessment Service	Independent Python/FastAPI service that receives contextual data via a REST API, applies a scoring function or model, and returns a risk score or classification.	Does not access Keycloak's database or admin API; does not issue or sign tokens; does not persist Keycloak identity data.

7.1 Interaction Flow

1. A user initiates an authentication request against a Keycloak-secured client application.
2. Keycloak performs standard authentication (credential validation, any configured authentication flows/MFA).
3. During the token-building phase, Keycloak invokes all configured Protocol Mappers for the client, including the custom Context-Aware Dynamic Token Mapper.
4. The custom mapper extracts contextual attributes available from the authentication/user session (IP address, timestamp, user-agent/device data, and any accessible login history).
5. The mapper issues a synchronous REST call to the external AI Risk Assessment Service, passing the collected context as a JSON payload.
6. The AI Risk Assessment Service evaluates the payload and returns a risk score or classification (e.g., LOW / MEDIUM / HIGH) and, optionally, recommended restrictions.
7. The mapper applies the corresponding claim transformation to the token representation currently being assembled: adjusting scopes/roles in the claim set and/or adding risk_level and restricted claims.
8. Keycloak signs and issues the resulting token; the modified claims exist only within that signed token and the current session, with no changes made to persisted user data.

A conceptual data-flow diagram, sequence diagram, and configuration schema for the mapper (admin-console attributes such as service endpoint, timeout, and risk-to-claim mapping rules) will be produced as part of the design documentation deliverable.

8. Expected Contributions

- A working demonstration of extending Keycloak through its supported Protocol Mapper framework, following Keycloak's own extension and packaging conventions (SPI provider registration, JAR deployment).
- A concrete, documented example of integrating a Java-based enterprise IAM platform with an independently deployable AI/analytics microservice over a REST interface.
- A practical implementation pattern for adaptive, session-scoped authorization that leaves persisted identity data untouched, which can inform future adaptive-access work on Keycloak or comparable IAM platforms.
- Reusable engineering artifacts: mapper configuration options, an example AI Risk Assessment Service contract (request/response schema), and deployment/test documentation.
- Improved personal and organizational understanding of secure JWT issuance, claim design, and the operational trade-offs of context-aware access control.

9. Scope and Limitations

The project is intentionally scoped as an architecture- and integration-focused engineering effort. The emphasis is on correctly extending Keycloak's extension points and cleanly integrating an external decision service, not on producing a sophisticated or production-hardened risk-scoring model.

In Scope	Out of Scope
Custom Protocol Mapper extending AbstractOIDCProtocolMapper, deployed as a Keycloak provider JAR.	Modifying Keycloak's core authentication engine, admin console, or database schema.
Collection of readily available session/context attributes (IP, timestamp, user-agent, basic login history if accessible via session notes/events).	Building a production-grade device-fingerprinting or behavioral-biometrics subsystem.
A minimal, purpose-built AI Risk Assessment Service exposing a REST scoring endpoint, using a simple rule-based or lightweight statistical/ML scoring function.	Research, training, or tuning of a sophisticated machine learning risk model; large-scale labeled datasets are not a project requirement.
Session-scoped, token-level claim transformation (scope/role suppression, risk_level and restricted claims).	Permanent alteration of user roles, groups, or attributes in the Keycloak database.
Functional testing in a local/self-hosted Keycloak development instance (e.g., Docker-based).	Multi-tenant, high-availability, or large-scale production deployment and load testing.
Basic resilience handling for the external service call (timeout, fallback to a default/safe claim state).	Comprehensive security hardening, formal threat modeling, or compliance certification of the risk service.

It is also acknowledged that, because the AI Risk Assessment Service is an external synchronous dependency, its availability and latency directly affect the token-issuance path; the design will therefore include an explicit, documented fallback behavior (e.g., defaulting to a conservative or unmodified claim set) in the event the service is unreachable or times out.

10. Technologies to be Used

Layer	Technology / Tooling
IAM Platform	Keycloak (open-source), deployed locally via Docker/Docker Compose for development and testing
Mapper Implementation	Java, Keycloak SPI (AbstractOIDCProtocolMapper and related token/session APIs), Maven for build and packaging
Token Standards	OAuth 2.0, OpenID Connect (OIDC), JSON Web Token (JWT)
AI Risk Service	Python 3, FastAPI, Uvicorn; a lightweight rule-based or simple statistical scoring function
Integration	REST/HTTP (JSON) between the Keycloak mapper and the AI Risk Assessment Service
Testing	JUnit for mapper unit tests; Postman/HTTP clients for API-level testing; manual end-to-end token inspection
Documentation	Markdown/Word technical documentation, sequence and architecture diagrams

11. Expected Deliverables

1. A technical design document describing Keycloak's Protocol Mapper framework and the proposed extension's architecture (component responsibilities, sequence diagram, configuration schema).
2. Source code for the custom Java Protocol Mapper, packaged as a deployable Keycloak provider JAR, with accompanying build instructions.
3. Source code for the external AI Risk Assessment Service (Python/FastAPI), including its API contract (request/response schema).
4. A local/demo deployment setup (e.g., Docker Compose) enabling end-to-end demonstration of the token modification flow.
5. Test cases and results demonstrating correct claim modification for varying risk classifications, and confirming that stored user data remains unmodified.
6. A final internship report summarizing the implementation, design decisions, challenges encountered, and recommendations for future work.

12. Conclusion

This project offers a focused, architecturally grounded opportunity to extend a production-grade open-source IAM platform in a way that is both practically useful and pedagogically rich. By working within Keycloak's own Protocol Mapper extension point, the project respects the platform's design boundaries while introducing meaningful new capability: session-scoped, context-aware authorization decisions communicated through token claims.

The clear separation of responsibilities between Keycloak, the custom Protocol Mapper, and the external AI Risk Assessment Service ensures that the resulting system remains modular, testable, and maintainable, and that the integrity of persisted identity data is never put at risk. The emphasis throughout is on sound IAM software architecture and secure token engineering, with the AI component deliberately kept lightweight so that the core learning outcomes, understanding Keycloak internals, extensibility patterns, and adaptive authorization design, remain the central focus of the internship.